ASSIGNMENT 01

Course: Design and Analysis of Algorithms
Program: B.Tech. AIML – 4th Semester
Session: Jan–June 2026
Name: Kunal vats Roll No: 2411062

1. What are the three main steps in a typical Divide and Conquer algorithm?

Divide and Conquer is an algorithm design paradigm that solves a problem by dividing it into smaller subproblems, solving those subproblems independently, and then combining their results to obtain the final solution. It is widely used in many efficient algorithms such as Merge Sort, Quick Sort, and Binary Search.

A typical Divide and Conquer algorithm consists of three main steps:

1. Divide

In this step, the main problem is divided into a number of smaller subproblems. These subproblems are usually similar to the original problem but are smaller in size. The division continues until the problem becomes simple enough to solve directly.

For example, in Merge Sort, an array is divided into two halves repeatedly until each subarray contains only one element.

2. Conquer

In the conquer step, the smaller subproblems are solved. If the subproblem is still large, the divide and conquer method is applied recursively until the base case is reached. The base case is the smallest instance of the problem that can be solved directly without further division.

3. Combine

After solving the subproblems, their solutions are combined to form the solution of the original problem. This step is important because it integrates the results obtained from each subproblem.

For example, in Merge Sort, the sorted subarrays are merged together to produce the final sorted array.

Thus, Divide, Conquer, and Combine are the three fundamental steps in the divide and conquer technique.

---

2. What is the role of recursion in Divide and Conquer algorithms?

Recursion plays a very important role in divide and conquer algorithms. Recursion is a programming technique in which a function calls itself in order to solve smaller instances of a problem.

In divide and conquer algorithms, recursion helps in repeatedly dividing the problem into smaller parts until the base condition is reached.

The role of recursion includes the following:

1. **Breaking the problem into smaller parts**
   Recursion allows the algorithm to repeatedly divide the problem into smaller subproblems. Each recursive call works on a smaller portion of the original input.
2. **Simplifying problem solving**
   Recursion makes the implementation of divide and conquer algorithms simpler and easier to understand because the same logic is applied repeatedly.
3. **Reaching the base case**
   Every recursive algorithm must have a base case. When the problem size becomes small enough, recursion stops and the result is returned.
4. **Combining solutions**
   After solving smaller subproblems recursively, the results are combined to produce the final solution.

For example, in **Binary Search**, recursion is used to search the element in either the left half or the right half of the array until the element is found or the search space becomes empty.

Thus, recursion is the mechanism that allows divide and conquer algorithms to work effectively.

---

3. Is Quick Sort and Merge Sort having Stable and In-Place sorting? Explain.

Sorting algorithms are often evaluated based on two important properties: **stability** and **in-place sorting**.

Stability

A sorting algorithm is said to be stable if elements with equal values maintain their relative order after sorting.

In-place Sorting

An in-place sorting algorithm sorts the elements without requiring significant extra memory.

Quick Sort

Quick Sort is generally considered an **in-place sorting algorithm** because it requires only a small amount of additional memory for recursion.

However, Quick Sort is **not stable**. During the partitioning process, equal elements may change their relative order, which means their original order is not preserved.

Merge Sort

Merge Sort is a **stable sorting algorithm**. When two elements have the same value, their relative order remains unchanged after sorting.

However, Merge Sort is **not an in-place algorithm** because it requires additional memory to merge the sorted subarrays.

Summary

Algorithm	Stable	In-Place
Quick Sort	No	Yes
Merge Sort	Yes	No

Therefore, Quick Sort is efficient in terms of memory usage, while Merge Sort maintains stability during sorting.

---

4. Explain Time Complexity of Strassen's Matrix Multiplication Algorithm.

Matrix multiplication is an important operation in computer science and mathematics. The traditional method of multiplying two matrices of size (n \times n) requires **$O(n^3)$** time.

Strassen's algorithm is an improved algorithm developed by **Volker Strassen** that reduces the number of multiplications required.

In the traditional method, 8 multiplications are required to multiply two matrices of size (2 \times 2). Strassen discovered that this can be reduced to **7 multiplications**, which reduces the overall computational complexity.

The recurrence relation for Strassen's algorithm is:

[
T(n) = 7T(n/2) + O(n^2)
]

Where:

- (7T(n/2)) represents the seven recursive multiplications
- (O(n^2)) represents the additional work needed for matrix addition and subtraction

Using the **Master Theorem**, the time complexity becomes:

[
T(n) = O(n^{\log_2 7})
]

Since:

[
\log_2 7 \approx 2.81
]

The final complexity is:

[
O(n^{2.81})
]

This is significantly faster than the traditional (O(n^3)) algorithm for large matrices.

---

5. Find Time Complexity of the Following Codes

(1)

```
int i, j, k = 0;

for (i = n / 2; i <= n; i++) {

   for (j = 2; j <= n; j = j * 2) {

      k = k + n / 2;

   }

}
```

Analysis

The outer loop runs from **n/2 to n**, which means it runs approximately **n/2 times**.

Therefore:

[
O(n)
]

The inner loop multiplies **j by 2** in each iteration:

[
2,4,8,16,...,n
]

This results in **$\log_2 n$ iterations**.

Therefore:

[
O(\log n)
]

Total Time Complexity:

[
O(n \log n)
]

---

(2)

```
int a = 0, i = N;

while (i > 0) {

   a += i;

   i /= 2;

}
```

Analysis

The value of **i** is divided by 2 in each iteration:

[
N, N/2, N/4, N/8 ...
]

The number of iterations becomes:

[
\log_2 N
]

Therefore, the time complexity is:

[
O(\log N)
]

---

(3)

```
int a = 0, b = 0;


for (i = 0; i < N; i++) {
   a = a + rand();
}


for (j = 0; j < M; j++) {
   b = b + rand();
}
```

Time Complexity

First loop executes **N times**:

[
O(N)
]

Second loop executes **M times**:

[
O(M)
]

Total Time Complexity:

[
O(N + M)
]

Space Complexity

Only a few variables are used.

[
O(1)
]

---

6. How is Time Complexity Measured?

Time complexity is a measure used to describe the efficiency of an algorithm. It represents the amount of time an algorithm takes to run as a function of the input size.

Time complexity is measured by counting the number of fundamental operations performed by the algorithm.

Instead of measuring actual execution time, which depends on hardware and programming language, theoretical analysis is used.

Time complexity is expressed using **asymptotic notation**, such as:

Big O Notation (O) – Represents the worst-case complexity.
Omega Notation (Ω) – Represents the best-case complexity.
Theta Notation (Θ) – Represents the average-case complexity.

Examples:

Linear Search → (O(n))
Binary Search → (O(\log n))
Merge Sort → (O(n \log n))

Thus, time complexity helps compare algorithms and determine which algorithm is more efficient.

---

7. Advantages and Disadvantages of Arrays

Arrays are one of the most commonly used data structures in computer programming. They store multiple elements of the same type in contiguous memory locations.

Advantages

1. **Fast access**
   Elements can be accessed quickly using an index.
2. **Simple implementation**
   Arrays are easy to use and implement.
3. **Efficient memory usage**
   Since elements are stored in contiguous memory, memory access is faster.
4. **Useful for implementing other data structures**
   Many data structures such as stacks and queues are implemented using arrays.

Disadvantages

1. **Fixed size**
   The size of an array must be defined in advance and cannot easily be changed.

2. **Insertion and deletion are costly**
   Inserting or deleting elements requires shifting other elements.
3. **Memory wastage**
   If the allocated size is larger than needed, memory is wasted.

---

8. Time Complexity of Linked List Operations

A linked list is a dynamic data structure in which elements are connected using pointers.

The time complexity of common operations in a linked list is:

Operation	Time Complexity
Access element	O(n)
Search element	O(n)
Insertion at beginning	O(1)
Insertion at end	O(n)
Deletion	O(n)

Linked lists provide flexibility in memory allocation and efficient insertion and deletion operations compared to arrays.

---

9. What is the Running Time of Strassen's Algorithm?

Strassen's algorithm is an optimized algorithm for matrix multiplication that reduces the number of multiplication operations.

The recurrence relation is:

[
T(n) = 7T(n/2) + O(n^2)
]

Using the Master Theorem, the running time becomes:

[
T(n) = O(n^{2.81})
]

Therefore, the running time of Strassen's matrix multiplication algorithm is approximately **$O(n^{2.81})$**, which is faster than the classical **$O(n^3)$** algorithm.

---

10. Worst Case and Average Case Complexity of Binary Search Using Recursion

Binary Search is an efficient searching algorithm used for sorted arrays. It works by repeatedly dividing the search interval into two halves.

If the target value is less than the middle element, the search continues in the left half. Otherwise, it continues in the right half.

Each recursive call reduces the number of elements by half.

Therefore, the number of steps required is:

[
\log_2 n
]

Worst Case Complexity

[
O(\log n)
]

This occurs when the element is found at the deepest level or not found in the array.

Average Case Complexity

[
O(\log n)
]

On average, the search also requires logarithmic time because the search space is reduced by half at each step.
